

Contents

Python Coding Interview Preparation Guide	2
Table of Contents	2
Introduction	2
How to Approach Coding Interviews	2
Problem-Solving Framework	3
Common Python Libraries and Built-in Functions	3
Collections Module	3
Heapq Module	4
Itertools Module	4
Built-in Functions	5
Other Useful Tools	5
Arrays and Hashing	6
Theory	6
Example Problems	7
Two Pointers and Sliding Window	9
Theory	9
Example Problems	10
Stack and Queue	13
Theory	13
Example Problems	14
Binary Search	16
Theory	16
Example Problems	18
Linked Lists	20
Theory	20
Example Problems	21
Trees	23
Theory	23
Example Problems	25
Heaps/Priority Queues	27
Theory	27
Example Problems	28
Dynamic Programming	31
Theory	31
Example Problems	32
Backtracking	35
Theory	35
Example Problems	36
Greedy Algorithms	39
Theory	39
Example Problems	40
Graphs	42
Theory	42
Example Problems	45

Intervals	48
Theory	48
Example Problems	49
Math and Geometry	51
Theory	51
Example Problems	53
Trie Data Structure	55
Theory	55
Example Problems	57
SQL for Analytics	60
Example Problems	60

Python Coding Interview Preparation Guide

Table of Contents

1. Introduction
2. Common Python Libraries and Built-in Functions
3. Arrays and Hashing
4. Two Pointers and Sliding Window
5. Stack and Queue
6. Binary Search
7. Linked Lists
8. Trees
9. Heaps/Priority Queues
10. Dynamic Programming
11. Backtracking
12. Greedy Algorithms
13. Graphs
14. Intervals
15. Math and Geometry
16. Trie Data Structure
17. SQL for Analytics

Introduction

How to Approach Coding Interviews

Coding interviews can be stressful, but with proper preparation and a systematic approach, you can perform well. Here are some key strategies:

1. **Understand the problem completely:**
 - Ask clarifying questions
 - Confirm your understanding with examples
 - Discuss edge cases early
2. **Think out loud:**

- Share your thought process
 - Explain trade-offs you're considering
 - Discuss alternative approaches
3. **Start with a simple solution:**
 - Begin with a brute force approach to show you can solve the problem
 - Optimize iteratively
 - Explain each optimization step clearly
 4. **Test your solution:**
 - Walk through your code with examples
 - Pay special attention to edge cases
 - Fix bugs methodically
 5. **Analyze complexity:**
 - Be prepared to discuss time and space complexity
 - Know the complexity of standard operations

Problem-Solving Framework

For most interview problems, follow this framework:

1. **Clarify the problem:**
 - What are the inputs and expected outputs?
 - What constraints exist?
 - Are there any performance requirements?
2. **Work through examples:**
 - Use simple examples to understand the problem
 - Try edge cases to identify potential issues
3. **Formulate an approach:**
 - Consider which data structures or algorithms might be appropriate
 - Think about the problem in terms of patterns you recognize
4. **Implement the solution:**
 - Write clean, readable code
 - Use descriptive variable names
 - Structure your code logically
5. **Test and debug:**
 - Test with your examples
 - Identify and fix any issues
 - Consider any optimizations

Common Python Libraries and Built-in Functions

Python's standard library contains many powerful tools that can simplify your solutions during coding interviews. Here are some of the most useful:

Collections Module

```
from collections import deque, Counter, defaultdict, OrderedDict
```

```

# deque: Efficient append/pop from both ends
queue = deque([1, 2, 3])
queue.append(4)           # Add to right: [1, 2, 3, 4]
queue.appendleft(0)       # Add to left: [0, 1, 2, 3, 4]
queue.pop()               # Remove from right: [0, 1, 2, 3]
queue.popleft()           # Remove from left: [1, 2, 3]

# Counter: Count occurrences
counts = Counter("mississippi") # {'i': 4, 's': 4, 'p': 2, 'm': 1}
most_common = counts.most_common(2) # [('i', 4), ('s', 4)]

# defaultdict: Dictionary with default values
word_groups = defaultdict(list)
words = ["eat", "tea", "tan", "ate", "nat", "bat"]
for word in words:
    sorted_word = "".join(sorted(word))
    word_groups[sorted_word].append(word)
# defaultdict(<class 'list'>, {'aet': ['eat', 'tea', 'ate'], 'ant': ['tan', 'nat'], 'abt':

# OrderedDict: Dictionary that remembers insertion order (less important since Python 3.7)
ordered = OrderedDict()
ordered['first'] = 1
ordered['second'] = 2

```

Heapq Module

```

import heapq

# Create a heap from a list
nums = [3, 1, 4, 1, 5, 9]
heapq.heapify(nums) # [1, 1, 4, 3, 5, 9]

# Push item to heap
heapq.heappush(nums, 2) # [1, 1, 2, 3, 5, 9, 4]

# Pop smallest item
smallest = heapq.heappop(nums) # smallest = 1, nums = [1, 3, 2, 4, 5, 9]

# Get n smallest/largest elements
smallest_three = heapq.nsmallest(3, [3, 1, 4, 1, 5, 9]) # [1, 1, 3]
largest_three = heapq.nlargest(3, [3, 1, 4, 1, 5, 9]) # [9, 5, 4]

```

Itertools Module

```

import itertools

```

```

# combinations: All possible ways to select k items
combinations = list(itertools.combinations([1, 2, 3, 4], 2))
# [(1, 2), (1, 3), (1, 4), (2, 3), (2, 4), (3, 4)]

# permutations: All possible arrangements of k items
permutations = list(itertools.permutations([1, 2, 3], 2))
# [(1, 2), (1, 3), (2, 1), (2, 3), (3, 1), (3, 2)]

# product: Cartesian product
product = list(itertools.product([1, 2], ['a', 'b']))
# [(1, 'a'), (1, 'b'), (2, 'a'), (2, 'b')]

# groupby: Group consecutive items
numbers = [1, 1, 1, 2, 2, 3, 3, 3, 3]
groups = [(k, list(g)) for k, g in itertools.groupby(numbers)]
# [(1, [1, 1, 1]), (2, [2, 2]), (3, [3, 3, 3, 3])]

```

Built-in Functions

```

# all() and any()
all([True, True, False]) # False
any([True, True, False]) # True

# map() and filter()
squared = list(map(lambda x: x**2, [1, 2, 3, 4])) # [1, 4, 9, 16]
evens = list(filter(lambda x: x % 2 == 0, [1, 2, 3, 4])) # [2, 4]

# zip(): Combine iterables
names = ["Alice", "Bob", "Charlie"]
ages = [25, 30, 35]
combined = list(zip(names, ages)) # [('Alice', 25), ('Bob', 30), ('Charlie', 35)]

# enumerate(): Get index and value
for i, name in enumerate(names):
    print(f"Index {i}: {name}")

# sorted() with key function
words = ["banana", "apple", "cherry"]
by_length = sorted(words, key=len) # ['apple', 'banana', 'cherry']

```

Other Useful Tools

```

# List Comprehensions
squares = [x**2 for x in range(10) if x % 2 == 0] # [0, 4, 16, 36, 64]

# Dictionary Comprehensions

```

```

square_map = {x: x**2 for x in range(5)} # {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}

# Set Comprehensions
even_squares = {x**2 for x in range(10) if x % 2 == 0} # {0, 4, 16, 36, 64}

# bisect: Binary search functions
import bisect
sorted_list = [1, 3, 5, 7, 9]
insert_point = bisect.bisect_left(sorted_list, 4) # 2
bisect.insort_left(sorted_list, 4) # [1, 3, 4, 5, 7, 9]

```

Arrays and Hashing

Theory

Arrays and hash tables (dictionaries in Python) are fundamental data structures used in many algorithms. Understanding when and how to use them effectively is crucial for solving many coding interview problems.

Arrays Arrays in Python are implemented as dynamic lists that automatically resize as elements are added or removed.

Key Operations and Complexity: - Access by index: $O(1)$ - Append/Pop from end: $O(1)$ amortized - Insert/Delete at beginning or middle: $O(n)$ - Search for element: $O(n)$ - Slice: $O(k)$ where k is the length of the slice

Common Array Patterns: 1. **Two-pass technique:** Process the array once to gather information, then process it again to solve the problem 2. **Prefix sums:** Pre-compute cumulative sums to enable $O(1)$ range sum queries 3. **Sorting:** Many problems become easier after sorting the array 4. **In-place modification:** Modify the array without using extra space

Hash Tables (Dictionaries) Hash tables provide efficient lookup, insertion, and deletion operations using a key-value mapping.

Key Operations and Complexity: - Lookup: $O(1)$ average, $O(n)$ worst-case - Insert/Delete: $O(1)$ average, $O(n)$ worst-case - Iterate: $O(n)$

Common Hashing Patterns: 1. **Counting elements:** Using a dictionary to count occurrences 2. **Grouping elements:** Mapping values to groups with similar properties 3. **Memorization:** Storing computed results for quick lookup 4. **Two-Sum pattern:** Finding pairs with specific relationships

When to Use Arrays and Hashing Use arrays when: - You need ordered elements with constant-time access by index - You're performing operations from the end of the collection - You need to efficiently iterate through all elements in order

Use hash tables when: - You need to quickly check if an element exists - You need to associate values with keys - You're looking for patterns or duplicates - You need to count occurrences of elements - You're implementing a cache

Identification Criteria: - The problem involves counting or grouping elements - You need to check for existence quickly - The problem mentions “find pairs” or relations between elements - You need to deduplicate elements - You need to track the frequency of elements

Example Problems

Easy: Two Sum Problem: Given an array of integers `nums` and an integer `target`, return indices of the two numbers such that they add up to `target`.

Approach: We can use a hash table to store the value and index of each number we've seen. For each number, we check if its complement (`target - num`) exists in the hash table.

```
def two_sum(nums: list[int], target: int) -> list[int]:
    seen = {} # Value -> Index

    for i, num in enumerate(nums):
        complement = target - num
        if complement in seen:
            return [seen[complement], i]
        seen[num] = i

    return [] # No solution found
```

Time Complexity: $O(n)$ - We traverse the array once Space Complexity: $O(n)$ - In the worst case, we store all elements in the hash table

Medium: Group Anagrams Problem: Given an array of strings `strs`, group the anagrams together. An anagram is a word formed by rearranging the letters of another word.

Approach: We'll use a hash table where the key is a sorted version of each string (so anagrams have the same key), and the value is a list of anagrams.

```
def group_anagrams(strs: list[str]) -> list[list[str]]:
    anagram_groups = {}

    for s in strs:
        # Sort the string to use as a key
        sorted_str = ''.join(sorted(s))

        # Add to existing group or create a new one
        if sorted_str in anagram_groups:
            anagram_groups[sorted_str].append(s)
```

```

    else:
        anagram_groups[sorted_str] = [s]

    # Return the groups
    return list(anagram_groups.values())

```

Time Complexity: $O(n * k \log k)$ where n is the number of strings and k is the maximum length of a string Space Complexity: $O(n * k)$ for storing all strings

Hard: Longest Consecutive Sequence **Problem:** Given an unsorted array of integers `nums`, return the length of the longest consecutive elements sequence.

Approach: We'll use a hash set for $O(1)$ lookups. For each number, we check if it's the start of a sequence (i.e., `num-1` is not in the set), then count upward as long as consecutive numbers exist.

```

def longest_consecutive(nums: list[int]) -> int:
    if not nums:
        return 0

    # Create a set for O(1) lookups
    num_set = set(nums)
    max_length = 0

    for num in num_set:
        # Only process numbers that are the start of a sequence
        if num - 1 not in num_set:
            current_num = num
            current_length = 1

            # Count consecutive numbers
            while current_num + 1 in num_set:
                current_num += 1
                current_length += 1

            max_length = max(max_length, current_length)

    return max_length

```

Time Complexity: $O(n)$ - Although we have nested loops, each number is only considered once as the start of a sequence Space Complexity: $O(n)$ - We store all numbers in a set

Two Pointers and Sliding Window

Theory

Two Pointers and Sliding Window are related techniques that use pointers to navigate through arrays or strings efficiently.

Two Pointers Technique The Two Pointers technique uses two pointers to traverse an array or string. These pointers can move toward each other, in the same direction at different speeds, or even start at different positions.

Common Two Pointer Patterns:

1. **Opposite Direction:** Pointers start from opposite ends and move toward each other
 - Used for: Reversing arrays, two sum on sorted arrays, container with most water
 - Time Complexity: Usually $O(n)$
2. **Same Direction:** Both pointers move in the same direction
 - Used for: Removing duplicates, finding specific patterns, merge operations
 - Time Complexity: Usually $O(n)$
3. **Fast and Slow:** One pointer moves faster than the other
 - Used for: Cycle detection, finding middle elements
 - Time Complexity: Usually $O(n)$

Sliding Window Technique The Sliding Window technique is a variation of the Two Pointers approach that maintains a “window” between two pointers, expanding or contracting it as needed.

Common Sliding Window Patterns:

1. **Fixed-Size Window:** Window size remains constant
 - Used for: Finding maximum/minimum sum or average in a fixed-size subarray
 - Time Complexity: $O(n)$
2. **Variable-Size Window:** Window expands or contracts based on conditions
 - Used for: Longest/shortest substring with certain properties
 - Time Complexity: Usually $O(n)$
3. **Dynamic Sliding Window with Hash Map/Counter:** Maintains a frequency count within the window
 - Used for: Substring problems involving character frequency
 - Time Complexity: $O(n)$

When to Use Two Pointers and Sliding Window Use Two Pointers when:

- The problem involves sorted arrays or specific positioning - You need to find

pairs or triplets with certain properties - You need to compare elements from different positions

Use Sliding Window when: - The problem involves subarrays or substrings - You need to find a continuous range with specific properties - The problem can be solved by expanding/contracting a range

Identification Criteria: - The problem involves arrays, strings, or linked lists - You need to find pairs, subarrays, or substrings with specific properties - The input is sorted (common for Two Pointers) - The problem asks for a maximum/minimum subarray or substring with constraints - The problem can be solved by processing consecutive elements

Time and Space Complexity: - Time Complexity: Usually $O(n)$ for both techniques - Space Complexity: Usually $O(1)$ as we typically only use a few extra variables

Example Problems

Easy: Valid Palindrome Problem: Given a string `s`, determine if it is a palindrome, considering only alphanumeric characters and ignoring case.

Approach: Use two pointers starting from the ends of the string and moving toward the center, skipping non-alphanumeric characters.

```
def is_palindrome(s: str) -> bool:
    # Convert to lowercase and initialize pointers
    s = s.lower()
    left, right = 0, len(s) - 1

    while left < right:
        # Skip non-alphanumeric characters
        if not s[left].isalnum():
            left += 1
            continue
        if not s[right].isalnum():
            right -= 1
            continue

        # Check if characters match
        if s[left] != s[right]:
            return False

        left += 1
        right -= 1

    return True
```

Time Complexity: $O(n)$ - Each character is visited at most once Space Complexity: $O(1)$ - We only use a constant amount of extra space

Medium: 3Sum Problem: Given an array of integers `nums`, return all triplets `[nums[i], nums[j], nums[k]]` such that $i \neq j, i \neq k, j \neq k$, and $nums[i] + nums[j] + nums[k] == 0$.

Approach: Sort the array first, then for each element, use two pointers to find the other two elements that sum to zero.

```
def three_sum(nums: list[int]) -> list[list[int]]:
    result = []
    nums.sort()

    for i in range(len(nums) - 2):
        # Skip duplicates
        if i > 0 and nums[i] == nums[i - 1]:
            continue

        # Use two pointers for the remaining array
        left, right = i + 1, len(nums) - 1

        while left < right:
            total = nums[i] + nums[left] + nums[right]

            if total < 0:
                left += 1
            elif total > 0:
                right -= 1
            else:
                result.append([nums[i], nums[left], nums[right]])

                # Skip duplicates
                while left < right and nums[left] == nums[left + 1]:
                    left += 1
                while left < right and nums[right] == nums[right - 1]:
                    right -= 1

                left += 1
                right -= 1

    return result
```

Time Complexity: $O(n^2)$ - Sorting is $O(n \log n)$, and the two-pointer approach is $O(n^2)$ Space Complexity: $O(1)$ - Excluding the output array

Hard: Minimum Window Substring Problem: Given two strings *s* and *t*, return the minimum window substring of *s* such that every character in *t* (including duplicates) is included in the window.

Approach: Use a sliding window with two pointers and a hash map to track character frequencies.

```
def min_window(s: str, t: str) -> str:
    from collections import Counter

    if not s or not t:
        return ""

    # Dictionary to keep count of characters in t
    target_count = Counter(t)
    required = len(target_count)

    # Dictionary to keep current window count
    window_count = {}
    formed = 0 # Number of unique characters matched

    # Pointers and result variables
    left = right = 0
    min_len = float('inf')
    result = ""

    while right < len(s):
        # Expand the window
        char = s[right]
        window_count[char] = window_count.get(char, 0) + 1

        # Check if we've matched all occurrences of this character
        if char in target_count and window_count[char] == target_count[char]:
            formed += 1

        # Try to contract the window from the left
        while left <= right and formed == required:
            char = s[left]

            # Update result if we find a smaller valid window
            if right - left + 1 < min_len:
                min_len = right - left + 1
                result = s[left:right + 1]

            # Remove the leftmost character
            window_count[char] -= 1
            if char in target_count and window_count[char] < target_count[char]:
                formed -= 1

            left += 1

        right += 1
```

```

        formed -= 1

    left += 1

    right += 1

    return result if min_len != float('inf') else ""

```

Time Complexity: $O(|s| + |t|)$ where $|s|$ and $|t|$ are the lengths of strings s and t

Space Complexity: $O(|s| + |t|)$ for the character frequency dictionaries

Stack and Queue

Theory

Stacks and queues are fundamental data structures that follow specific principles for adding and removing elements.

Stack A stack follows the Last-In-First-Out (LIFO) principle, where the last element added is the first one removed.

Key Operations and Complexity: - Push (add to top): $O(1)$ - Pop (remove from top): $O(1)$ - Peek (view top element): $O(1)$

Python Implementation:

```

# Using a list (built-in)
stack = []
stack.append(1) # Push
stack.append(2)
top = stack.pop() # Pop (returns 2)

# Using collections.deque (more efficient for large stacks)
from collections import deque
stack = deque()
stack.append(1) # Push
stack.append(2)
top = stack.pop() # Pop (returns 2)

```

Queue A queue follows the First-In-First-Out (FIFO) principle, where the first element added is the first one removed.

Key Operations and Complexity: - Enqueue (add to back): $O(1)$ - Dequeue (remove from front): $O(1)$ - Peek (view front element): $O(1)$

Python Implementation:

```

# Using collections.deque (efficient)
from collections import deque

```

```

queue = deque()
queue.append(1)      # Enqueue
queue.append(2)
front = queue.popleft() # Dequeue (returns 1)

# Using queue.Queue (thread-safe)
from queue import Queue
q = Queue()
q.put(1) # Enqueue
q.put(2)
front = q.get() # Dequeue (returns 1)

```

Monotonic Stack/Queue A monotonic stack/queue maintains elements in strictly increasing or decreasing order.

Monotonic Stack Applications: - Next/previous greater/smaller element problems - Largest rectangle in histogram - Maximum area under skyline

When to Use Stacks and Queues Use stacks when: - You need to process elements in reverse order - You're implementing a recursive algorithm iteratively - You need to track function calls (call stack) - You're parsing expressions (e.g., parentheses matching) - You need to implement undo/redo functionality

Use queues when: - You need to process elements in order of arrival - You're implementing breadth-first search - You're managing tasks in the order they were received - You need a buffer

Identification Criteria: - The problem involves matching pairs or parentheses - You need to track the most recent elements - The problem requires handling nested structures - You need to implement a breadth-first traversal - You need to validate syntax or expressions - The problem involves tracking a "maximum so far" in a specific direction

Time and Space Complexity: - Time Complexity: Usually $O(n)$ as each element is pushed and popped at most once - Space Complexity: $O(n)$ in the worst case when all elements are stored

Example Problems

Easy: Valid Parentheses Problem: Given a string `s` containing just the characters `('', ')', '{', '}', '['` and `']'`, determine if the input string is valid. An input string is valid if: 1. Open brackets must be closed by the same type of brackets. 2. Open brackets must be closed in the correct order.

Approach: Use a stack to keep track of opening brackets. When we encounter a closing bracket, check if it matches the most recent opening bracket.

```

def is_valid(s: str) -> bool:
    stack = []

```

```

bracket_map = {'(': ')', '[': ']', '{': '}', ' ': ' '}

for char in s:
    # If it's an opening bracket, push to stack
    if char not in bracket_map:
        stack.append(char)
    # If it's a closing bracket, check if it matches the top of the stack
    else:
        if not stack or stack.pop() != bracket_map[char]:
            return False

# Stack should be empty if all brackets are matched
return len(stack) == 0

```

Time Complexity: $O(n)$ - We scan the string once Space Complexity: $O(n)$ - In the worst case, the stack could contain all opening brackets

Medium: Daily Temperatures Problem: Given an array of daily temperatures, return an array such that, for each day in the input, tells you how many days you would have to wait until a warmer temperature.

Approach: Use a monotonic decreasing stack to keep track of temperatures. When we find a higher temperature, pop elements from the stack and calculate the difference.

```

def daily_temperatures(temperatures: list[int]) -> list[int]:
    n = len(temperatures)
    result = [0] * n
    stack = [] # Stack of indices

    for i in range(n):
        # While current temperature is higher than temperature at stack top
        while stack and temperatures[i] > temperatures[stack[-1]]:
            prev_idx = stack.pop()
            result[prev_idx] = i - prev_idx

        stack.append(i)

    return result

```

Time Complexity: $O(n)$ - Each index is pushed and popped at most once Space Complexity: $O(n)$ - In the worst case, the stack could contain all indices

Hard: Largest Rectangle in Histogram Problem: Given an array of integers `heights` representing the histogram's bar height where the width of each bar is 1, return the area of the largest rectangle in the histogram.

Approach: Use a monotonic increasing stack to find the previous and next smaller elements for each bar, which determines the width of the rectangle.

```
def largest_rectangle_area(heights: list[int]) -> int:
    stack = [] # Stack of indices
    max_area = 0
    n = len(heights)

    for i in range(n + 1):
        # Use height 0 as sentinel at the end
        current_height = heights[i] if i < n else 0

        # Pop and calculate areas when we find a shorter bar
        while stack and current_height < heights[stack[-1]]:
            height = heights[stack.pop()]
            width = i if not stack else i - stack[-1] - 1
            max_area = max(max_area, height * width)

        stack.append(i)

    return max_area
```

Time Complexity: $O(n)$ - Each bar is pushed and popped at most once
 Space Complexity: $O(n)$ - In the worst case, the stack could contain all indices

Binary Search

Theory

Binary search is a divide-and-conquer algorithm that efficiently finds an element in a sorted array by repeatedly dividing the search space in half.

Key Principles: 1. The array must be sorted 2. In each step, compare the target with the middle element 3. If target equals middle element, return the index 4. If target is less than middle, search the left half 5. If target is greater than middle, search the right half

Basic Implementation:

```
def binary_search(nums: list[int], target: int) -> int:
    left, right = 0, len(nums) - 1

    while left <= right:
        mid = (left + right) // 2 # Or mid = left + (right - left) // 2 to avoid overflow

        if nums[mid] == target:
            return mid
        elif nums[mid] < target:
            left = mid + 1
```



```

    else:
        right = mid - 1

    return -1 # Target not found

```

Common Variations:

1. Finding the leftmost occurrence:

```

def binary_search_leftmost(nums: list[int], target: int) -> int:
    left, right = 0, len(nums) - 1
    result = -1

    while left <= right:
        mid = (left + right) // 2

        if nums[mid] == target:
            result = mid # Found a match, but continue searching left
            right = mid - 1
        elif nums[mid] < target:
            left = mid + 1
        else:
            right = mid - 1

    return result

```

2. Finding the rightmost occurrence:

```

def binary_search_rightmost(nums: list[int], target: int) -> int:
    left, right = 0, len(nums) - 1
    result = -1

    while left <= right:
        mid = (left + right) // 2

        if nums[mid] == target:
            result = mid # Found a match, but continue searching right
            left = mid + 1
        elif nums[mid] < target:
            left = mid + 1
        else:
            right = mid - 1

    return result

```

3. Binary search on answer space: Sometimes binary search can be applied to problems that don't involve arrays but have a range of possible answers.

When to Use Binary Search Use binary search when: - The input array is sorted or can be sorted - You need to find an element in a sorted array - You need to find an insertion point or boundary - You need to find a value that satisfies a specific condition - The problem has a monotonic property (increasing or decreasing)

Identification Criteria: - The problem mentions a sorted array - You need to find a specific value or point efficiently - The problem asks for an “optimum” within a range - The problem can be reduced to finding a threshold or boundary point - The problem involves finding a minimum/maximum value that satisfies a condition

Time and Space Complexity: - Time Complexity: $O(\log n)$ - The search space is halved in each step - Space Complexity: $O(1)$ for iterative implementation, $O(\log n)$ for recursive implementation

Example Problems

Easy: Binary Search Problem: Given a sorted array of integers `nums` and an integer `target`, return the index of `target` if it exists in `nums`, or -1 if it does not exist.

Approach: Use the standard binary search algorithm on the sorted array.

```
def search(nums: list[int], target: int) -> int:
    left, right = 0, len(nums) - 1

    while left <= right:
        mid = left + (right - left) // 2

        if nums[mid] == target:
            return mid
        elif nums[mid] < target:
            left = mid + 1
        else:
            right = mid - 1

    return -1
```

Time Complexity: $O(\log n)$ - The search space is halved in each step
Space Complexity: $O(1)$ - We only use a constant amount of extra space

Medium: Search in Rotated Sorted Array Problem: Given a rotated sorted array `nums` and an integer `target`, return the index of `target` if it exists in the array, or -1 if it does not exist.

Approach: We need to modify the binary search to handle the rotation. First, find which half of the array is sorted, then check if the target lies in that sorted half.

```

def search(nums: list[int], target: int) -> int:
    left, right = 0, len(nums) - 1

    while left <= right:
        mid = left + (right - left) // 2

        if nums[mid] == target:
            return mid

        # Check if left half is sorted
        if nums[left] <= nums[mid]:
            if nums[left] <= target < nums[mid]:
                right = mid - 1 # Target is in the left sorted half
            else:
                left = mid + 1 # Target is in the right half
        # Right half is sorted
        else:
            if nums[mid] < target <= nums[right]:
                left = mid + 1 # Target is in the right sorted half
            else:
                right = mid - 1 # Target is in the left half

    return -1

```

Time Complexity: $O(\log n)$ - We're still performing binary search
 Space Complexity: $O(1)$ - We only use a constant amount of extra space

Hard: Find Minimum in Rotated Sorted Array II **Problem:** Suppose an array of length n sorted in ascending order is rotated between 1 and n times. Given the sorted rotated array `nums` that may contain duplicates, return the minimum element of this array.

Approach: Use binary search, but when we can't determine which side the minimum is on (due to duplicates), we need to linearly eliminate one element.

```

def find_min(nums: list[int]) -> int:
    left, right = 0, len(nums) - 1

    while left < right:
        mid = left + (right - left) // 2

        if nums[mid] > nums[right]:
            left = mid + 1 # Minimum is in the right half
        elif nums[mid] < nums[right]:
            right = mid # Minimum is in the left half or at mid
        else:
            # Can't determine which side, so reduce the search space by 1
            right = right - 1

```

```
right -= 1
```

```
return nums[left]
```

Time Complexity: $O(\log n)$ in the average case, $O(n)$ in the worst case (when all elements are the same) Space Complexity: $O(1)$ - We only use a constant amount of extra space

Linked Lists

Theory

A linked list is a linear data structure where elements are stored in nodes, and each node points to the next node in the sequence.

Types of Linked Lists:

1. **Singly Linked List:** Each node has data and a reference to the next node
2. **Doubly Linked List:** Each node has data and references to both next and previous nodes
3. **Circular Linked List:** Last node points back to the first node

Python Implementation:

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next
```

Key Operations and Complexity:

- Access by index: $O(n)$
- Insert/Delete at beginning: $O(1)$
- Insert/Delete at end: $O(n)$ for singly linked list, $O(1)$ for doubly linked list
- Insert/Delete in the middle: $O(n)$ to find the position, $O(1)$ to perform the operation

Common Linked List Patterns:

1. **Two Pointers:** Fast and slow pointers, used for cycle detection, finding middle, etc.
2. **Dummy Head:** Creating a dummy head node to simplify operations on the list
3. **Reverse Linked List:** In-place reversal by changing links
4. **Merge Lists:** Combining multiple lists with specific criteria

When to Use Linked Lists Use linked lists when: - You need frequent insertions/deletions - You don't need random access - Memory allocation needs to be dynamic - You're implementing a queue, stack, or hash table (chaining)

Identification Criteria: - The problem explicitly involves linked lists - You need to manipulate individual elements without affecting others - The data structure needs to grow or shrink dynamically - You need to perform frequent insertions/deletions - You need to detect cycles or find mid-points efficiently

Time and Space Complexity: - Time Complexity: Operations typically range from $O(1)$ to $O(n)$ - Space Complexity: Usually $O(1)$ for operations, $O(n)$ for storing the list

Example Problems

Easy: Reverse Linked List Problem: Given the head of a singly linked list, reverse the list and return the reversed list.

Approach: Iteratively reverse the links of each node. Keep track of the previous, current, and next nodes.

```
def reverse_list(head: ListNode) -> ListNode:
    prev = None
    current = head

    while current:
        next_temp = current.next # Store next node
        current.next = prev      # Reverse the link
        prev = current          # Move prev forward
        current = next_temp      # Move current forward

    # After the loop, prev will point to the new head
    return prev
```

Time Complexity: $O(n)$ - We visit each node once Space Complexity: $O(1)$ - We only use a constant amount of extra space

Medium: Remove Nth Node From End of List Problem: Given the head of a linked list, remove the nth node from the end of the list and return its head.

Approach: Use two pointers, with the first pointer n nodes ahead of the second. When the first reaches the end, the second will be at the node before the one to be deleted.

```
def remove_nth_from_end(head: ListNode, n: int) -> ListNode:
    # Create a dummy node to handle edge cases
    dummy = ListNode(0)
    dummy.next = head
```

```

first = second = dummy

# Advance first pointer by n+1 steps
for _ in range(n + 1):
    if not first:
        break
    first = first.next

# Move both pointers until first reaches the end
while first:
    first = first.next
    second = second.next

# Remove the nth node
second.next = second.next.next

return dummy.next

```

Time Complexity: $O(n)$ - We traverse the list once Space Complexity: $O(1)$ - We only use a constant amount of extra space

Hard: Merge k Sorted Lists **Problem:** Given an array of k linked-lists lists, each linked-list is sorted in ascending order. Merge all the linked-lists into one sorted linked-list and return it.

Approach: Use a min-heap to efficiently merge the lists by always selecting the smallest element from all list heads.

```

import heapq

def merge_k_lists(lists: list[ListNode]) -> ListNode:
    # Create a dummy head
    dummy = ListNode(0)
    current = dummy

    # Create a min heap
    # We need to use a counter as a tiebreaker since ListNode objects aren't comparable
    from itertools import count
    counter = count()
    heap = []

    # Add the first node from each list to the heap
    for head in lists:
        if head:
            # Store (value, counter, node) in the heap
            heapq.heappush(heap, (head.val, next(counter), head))

```

```

# Process the heap
while heap:
    # Get the smallest value
    val, _, node = heapq.heappop(heap)

    # Add it to our result list
    current.next = node
    current = current.next

    # If there are more nodes in this list, add the next one to the heap
    if node.next:
        heapq.heappush(heap, (node.next.val, next(counter), node.next))

return dummy.next

```

Time Complexity: $O(N \log k)$ where N is the total number of nodes and k is the number of lists
Space Complexity: $O(k)$ for the priority queue

Trees

Theory

Trees are hierarchical data structures consisting of nodes connected by edges, with one node designated as the root.

Types of Trees:

Tree Type	Description	Properties	Common Applications
Binary Tree	Each node has at most 2 children	Simple structure	Expression trees, decision trees
Binary Search Tree (BST)	Left child < node < right child	In-order traversal yields sorted sequence	Searching, sorting, dynamic sets
AVL Tree	Self-balancing BST	Height-balanced, max height difference of 1	Efficient lookups, balanced operations
Red-Black Tree	Self-balancing BST	Guaranteed $O(\log n)$ operations	Implement maps, sets in standard libraries
B-Tree	Balanced m-way search tree	Multiple keys per node, used in databases	Disk storage, databases, file systems

Tree Type	Description	Properties	Common Applications
Trie	Each node stores one character	Path from root represents string	Dictionary, autocomplete, string search
Heap	Complete binary tree with heap property	Root is min/max of all nodes	Priority queues, graph algorithms
Segment Tree	Binary tree for range queries	Each node represents interval	Range queries, lazy propagation

Basic Tree Node Definition:

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right
```

Tree Traversals:

1. **Pre-order:** Root $\rightarrow$ Left $\rightarrow$ Right

```
def preorder(root):
    if not root:
        return
    print(root.val) # Process root
    preorder(root.left) # Process left subtree
    preorder(root.right) # Process right subtree
```

2. **In-order:** Left $\rightarrow$ Root $\rightarrow$ Right

```
def inorder(root):
    if not root:
        return
    inorder(root.left) # Process left subtree
    print(root.val) # Process root
    inorder(root.right) # Process right subtree
```

3. **Post-order:** Left $\rightarrow$ Right $\rightarrow$ Root

```
def postorder(root):
    if not root:
        return
    postorder(root.left) # Process left subtree
    postorder(root.right) # Process right subtree
    print(root.val) # Process root
```


4. Level-order (BFS): Process nodes level by level

```
from collections import deque

def levelorder(root):
    if not root:
        return
    queue = deque([root])
    while queue:
        node = queue.popleft()
        print(node.val)  # Process node
        if node.left:
            queue.append(node.left)
        if node.right:
            queue.append(node.right)
```

When to Use Trees Use trees when: - You need hierarchical data representation - You need efficient search, insert, and delete operations - You need to represent sorted data - You need to quickly find common ancestors or paths

Identification Criteria: - The problem explicitly mentions trees or hierarchical structures - The problem involves traversing or searching hierarchical data - The problem requires efficient range queries - The problem deals with paths, ancestors, or subtrees - The problem can be solved recursively with a tree-like substructure

Time and Space Complexity: - Most BST operations: $O(\log n)$ average, $O(n)$ worst (if unbalanced) - Tree traversal: $O(n)$ time, $O(h)$ space (where h is the height) - Balanced tree operations: $O(\log n)$ time guaranteed

Example Problems

Easy: Maximum Depth of Binary Tree Problem: Given the root of a binary tree, return its maximum depth (the number of nodes along the longest path from the root node down to the farthest leaf node).

Approach: Use recursion to find the depth of the left and right subtrees, then return the maximum plus one.

```
def max_depth(root: TreeNode) -> int:
    if not root:
        return 0

    left_depth = max_depth(root.left)
    right_depth = max_depth(root.right)

    return max(left_depth, right_depth) + 1
```

Time Complexity: $O(n)$ - We visit each node once Space Complexity: $O(h)$ - Where h is the height of the tree (for the recursion stack)

Medium: Validate Binary Search Tree Problem: Given the root of a binary tree, determine if it is a valid binary search tree (BST).

Approach: Use recursion with a range validation. Each node's value must be within a valid range based on its position in the tree.

```
def is_valid_bst(root: TreeNode) -> bool:
    def validate(node, low=float('-inf'), high=float('inf')):
        if not node:
            return True

        # Check if current node's value is within range
        if node.val <= low or node.val >= high:
            return False

        # Validate left subtree (must be less than current node)
        # Validate right subtree (must be greater than current node)
        return validate(node.left, low, node.val) and validate(node.right, node.val, high)

    return validate(root)
```

Time Complexity: $O(n)$ - We visit each node once Space Complexity: $O(h)$ - Where h is the height of the tree

Hard: Binary Tree Maximum Path Sum Problem: Given the root of a binary tree, return the maximum path sum. A path is a sequence of nodes where no node is visited more than once, and the path does not need to pass through the root.

Approach: Use recursion to compute the maximum path sum for each subtree, tracking the global maximum path sum.

```
def max_path_sum(root: TreeNode) -> int:
    # Initialize global maximum
    max_sum = float('-inf')

    def max_gain(node):
        nonlocal max_sum
        if not node:
            return 0

        # Get max path sum from left and right subtrees
        # If negative, use 0 (don't include that subtree)
        left_gain = max(max_gain(node.left), 0)
        right_gain = max(max_gain(node.right), 0)
```

```

    # Compute max path sum through the current node
    current_path_sum = node.val + left_gain + right_gain

    # Update global maximum
    max_sum = max(max_sum, current_path_sum)

    # Return max path sum starting from current node
    # (can only use one subtree for path going up)
    return node.val + max(left_gain, right_gain)

max_gain(root)
return max_sum

```

Time Complexity: $O(n)$ - We visit each node once Space Complexity: $O(h)$ - Where h is the height of the tree

Heaps/Priority Queues

Theory

A heap is a complete binary tree where every parent node has a value that satisfies a specific ordering property relative to its children. In a min-heap, each parent is less than or equal to its children. In a max-heap, each parent is greater than or equal to its children.

Key Properties:

- Complete binary tree structure
- Heap property (min-heap or max-heap)
- Root element is always the minimum (min-heap) or maximum (max-heap)
- Efficient insertion and extraction

Python Implementation: Python's built-in `heapq` module implements a min-heap:

```

import heapq

# Create a heap
heap = []
heapq.heapify(heap) # Transform list into a heap (in-place)

# Add elements
heapq.heappush(heap, 5)
heapq.heappush(heap, 3)
heapq.heappush(heap, 7)

```

```

# Remove smallest element
smallest = heapq.heappop(heap) # Returns 3

# Get smallest without removing
smallest = heap[0] # Peek at the smallest element

# For a max-heap, negate the values
max_heap = []
heapq.heappush(max_heap, -5) # Add -5
max_largest = -heapq.heappop(max_heap) # Get the maximum value (5)

```

Key Operations and Complexity:

- Build Heap: $O(n)$
- Insert: $O(\log n)$
- Extract Min/Max: $O(\log n)$
- Peek at Min/Max: $O(1)$
- Heapify (percolate down): $O(\log n)$

Common Heap Applications:

1. **Priority Queue:** Efficiently get the highest/lowest priority element
2. **Dijkstra's Algorithm:** Find shortest paths in a graph
3. **Heap Sort:** Sorting in $O(n \log n)$ time
4. **Median Maintenance:** Find median in a stream of numbers
5. **K-th Element:** Find k-th smallest/largest element

When to Use Heaps Use heaps when: - You need quick access to the minimum/maximum element - You need to efficiently get the k smallest/largest elements - You need to implement a priority queue - You're dealing with a data stream and need to maintain sorted behavior

Identification Criteria: - The problem mentions "top k" or "k smallest/largest" elements - You need to efficiently find the minimum/maximum element repeatedly - The problem involves scheduling or prioritizing tasks - The problem involves a "running median" or other order statistics - You need to merge multiple sorted sequences efficiently

Time and Space Complexity: - Space Complexity: $O(n)$ to store the heap - Most operations: $O(\log n)$ time complexity - Building a heap: $O(n)$ time complexity

Example Problems

Easy: Kth Largest Element in a Stream Problem: Design a class to find the kth largest element in a stream of numbers.

Approach: Maintain a min-heap of size k. When adding a new element, add it to the heap and remove the smallest if the heap size exceeds k.

```
import heapq

class KthLargest:
    def __init__(self, k: int, nums: list[int]):
        self.k = k
        self.heap = []

        # Initialize the heap with the first k elements
        for num in nums:
            self.add(num)

    def add(self, val: int) -> int:
        # Add the new value to the heap
        heapq.heappush(self.heap, val)

        # If heap size exceeds k, remove the smallest
        if len(self.heap) > self.k:
            heapq.heappop(self.heap)

        # Return the kth largest (smallest in our min-heap)
        return self.heap[0]
```

Time Complexity: - Initialization: $O(n \log k)$ - For adding n elements - Add operation: $O(\log k)$ - Heap operations with k elements Space Complexity: $O(k)$
 - We maintain a heap of size k

Medium: Top K Frequent Elements Problem: Given an integer array nums and an integer k, return the k most frequent elements.

Approach: Use a hash map to count frequencies, then use a heap to find the k most frequent elements.

```
from collections import Counter
import heapq

def top_k_frequent(nums: list[int], k: int) -> list[int]:
    # Count frequencies
    counter = Counter(nums)

    # Use a min-heap to keep track of the k most frequent elements
    # Store elements as (-frequency, number) for min-heap to act as max-heap
    heap = []

    for num, freq in counter.items():
```

```

        # Push to heap
        heapq.heappush(heap, (freq, num))

        # If heap size exceeds k, remove the least frequent
        if len(heap) > k:
            heapq.heappop(heap)

        # Extract numbers from heap in reverse order (most frequent first)
        result = [num for _, num in sorted(heap, reverse=True)]

    return result

```

Time Complexity: $O(n \log k)$ - Where n is the length of the input array
 Space Complexity: $O(n)$ - For the counter and heap

Hard: Find Median from Data Stream **Problem:** Design a data structure that supports adding integer numbers and calculating the median.

Approach: Maintain two heaps - a max-heap for the lower half and a min-heap for the upper half. Ensure they're balanced to easily find the median.

```

import heapq

class MedianFinder:
    def __init__(self):
        self.small = [] # Max-heap for the lower half
        self.large = [] # Min-heap for the upper half

    def add_num(self, num: int) -> None:
        # Add to the appropriate heap
        if len(self.small) == len(self.large):
            # If equal, add to large heap first, then move smallest to small heap
            heapq.heappush(self.large, num)
            smallest_large = heapq.heappop(self.large)
            heapq.heappush(self.small, -smallest_large) # Negate for max-heap
        else:
            # If unequal, add to small heap first, then move largest to large heap
            heapq.heappush(self.small, -num) # Negate for max-heap
            largest_small = -heapq.heappop(self.small)
            heapq.heappush(self.large, largest_small)

    def find_median(self) -> float:
        if len(self.small) == len(self.large):
            # If equal, median is average of the two middle elements
            return (-self.small[0] + self.large[0]) / 2
        else:
            # If unequal, median is the top of the small heap

```

```
return -self.small[0]
```

Time Complexity: - add_num: $O(\log n)$ - Due to heap operations - find_median: $O(1)$ - Direct access to the top elements Space Complexity: $O(n)$ - For storing all the elements in the heaps

Dynamic Programming

Theory

Dynamic Programming (DP) is a technique for solving complex problems by breaking them down into simpler subproblems and storing the solutions to avoid redundant calculations.

Key Principles:

1. **Optimal Substructure:** The optimal solution to the problem can be constructed from optimal solutions of its subproblems
2. **Overlapping Subproblems:** The problem can be broken down into subproblems which are reused multiple times

Two Main Approaches:

1. **Top-Down (Memoization):**
 - Start with the original problem
 - Break it down recursively
 - Store results of subproblems in a cache (usually a dictionary)
 - Check the cache before computing solutions
2. **Bottom-Up (Tabulation):**
 - Start with the smallest subproblems
 - Iteratively build up solutions
 - Store results in a table (usually an array)
 - Combine solutions to solve the original problem

Common DP Patterns:

1. **1D State:** Problems where state depends on previous states
 - Fibonacci numbers, climbing stairs, house robber
2. **2D State:** Problems involving sequences or grids
 - Longest common subsequence, edit distance, grid traversal
3. **State with Multiple Variables:** Problems with multiple dimensions of state
 - Knapsack problem, coin change with limited coins
4. **State Compression:** Using bits to represent states
 - Traveling salesman problem, subset problems

When to Use Dynamic Programming Use dynamic programming when: - The problem asks for optimization (min/max/longest/shortest) - The problem can be broken into overlapping subproblems - The problem asks for the number of ways to do something - The problem involves making a sequence of choices

Identification Criteria: - The problem involves optimization (min/max/best) - You notice overlapping subproblems when solving recursively - The problem asks for counting the number of ways - The problem can be solved using recursion, but is inefficient - The problem involves sequences, arrays, or strings with optimal solutions

Time and Space Complexity: - Time Complexity: Usually $O(n^2)$ or $O(n*m)$ depending on the state dimensions - Space Complexity: Usually the same as time complexity for tabulation, can be less for memoization

Example Problems

Easy: Climbing Stairs Problem: You are climbing a staircase. It takes n steps to reach the top. Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top?

Approach 1: Top-Down (Memoization)

```
def climb_stairs(n: int) -> int:
    # Initialize memoization cache
    memo = {}

    def dp(i: int) -> int:
        # Base cases
        if i == 0:
            return 1
        if i < 0:
            return 0

        # Check if already computed
        if i in memo:
            return memo[i]

        # Compute and store
        memo[i] = dp(i - 1) + dp(i - 2)
        return memo[i]

    return dp(n)
```

Approach 2: Bottom-Up (Tabulation)

```
def climb_stairs(n: int) -> int:
    if n <= 2:
        return n
```



```

# Initialize table
dp = [0] * (n + 1)
dp[1] = 1 # 1 way to climb 1 step
dp[2] = 2 # 2 ways to climb 2 steps

# Fill table bottom-up
for i in range(3, n + 1):
    dp[i] = dp[i - 1] + dp[i - 2]

return dp[n]

```

Time Complexity: $O(n)$ - We compute each step once Space Complexity: $O(n)$ - For the memoization table or DP array

Medium: Coin Change Problem: You are given coins of different denominations and a total amount of money. Write a function to compute the fewest number of coins needed to make up that amount.

Approach 1: Top-Down (Memoization)

```

def coin_change(coins: list[int], amount: int) -> int:
    # Initialize memoization cache
    memo = {}

    def dp(remaining: int) -> int:
        # Base cases
        if remaining == 0:
            return 0
        if remaining < 0:
            return float('inf')

        # Check if already computed
        if remaining in memo:
            return memo[remaining]

        # Try each coin and take the minimum
        min_coins = float('inf')
        for coin in coins:
            result = dp(remaining - coin)
            if result != float('inf'):
                min_coins = min(min_coins, result + 1)

        # Store and return
        memo[remaining] = min_coins
        return min_coins

```

```

result = dp(amount)
return result if result != float('inf') else -1

```

Approach 2: Bottom-Up (Tabulation)

```

def coin_change(coins: list[int], amount: int) -> int:
    # Initialize dp array with amount + 1 (impossible value)
    dp = [amount + 1] * (amount + 1)
    dp[0] = 0 # Base case: 0 coins needed to make amount 0

    # Fill dp array bottom-up
    for i in range(1, amount + 1):
        for coin in coins:
            if coin <= i:
                dp[i] = min(dp[i], dp[i - coin] + 1)

    return dp[amount] if dp[amount] <= amount else -1

```

Time Complexity: $O(\text{amount} * n)$ where n is the number of coins
 Space Complexity: $O(\text{amount})$ for the dp array

Hard: Edit Distance Problem: Given two strings `word1` and `word2`, return the minimum number of operations (insert, delete, replace) required to convert `word1` to `word2`.

Approach: Bottom-Up (Tabulation)

```

def min_distance(word1: str, word2: str) -> int:
    m, n = len(word1), len(word2)

    # Initialize dp table (m+1 x n+1)
    dp = [[0] * (n + 1) for _ in range(m + 1)]

    # Base cases: empty strings
    for i in range(m + 1):
        dp[i][0] = i # Delete i characters to match empty string

    for j in range(n + 1):
        dp[0][j] = j # Insert j characters to match target

    # Fill dp table
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if word1[i - 1] == word2[j - 1]:
                # Characters match, no operation needed
                dp[i][j] = dp[i - 1][j - 1]
            else:
                # Minimum of three operations:

```

```

# 1. Replace: dp[i-1][j-1] + 1
# 2. Delete: dp[i-1][j] + 1
# 3. Insert: dp[i][j-1] + 1
dp[i][j] = 1 + min(dp[i - 1][j - 1], dp[i - 1][j], dp[i][j - 1])

return dp[m][n]

```

Time Complexity: $O(m * n)$ where m and n are the lengths of the strings
Space Complexity: $O(m * n)$ for the dp table

Backtracking

Theory

Backtracking is an algorithmic technique for solving problems recursively by trying to build a solution incrementally, and abandoning a solution (“backtracking”) as soon as it determines that the current path cannot lead to a valid solution.

Key Principles:

1. **Recursive Exploration:** Try different choices recursively
2. **Constraint Checking:** Check if the current state violates any constraints
3. **Pruning:** Abandon paths that cannot lead to valid solutions
4. **State Management:** Save and restore state when backtracking

Common Backtracking Template:

```

def backtrack(state, choices, result):
    # Check if solution is complete
    if is_solution(state):
        result.append(state.copy()) # Add a copy of the current solution
        return

    # Try each possible choice
    for choice in choices:
        # Check if valid choice
        if is_valid(state, choice):
            # Make the choice
            state.add(choice)

            # Recurse with updated state
            backtrack(state, choices, result)

            # Undo the choice (backtrack)
            state.remove(choice)

```

Common Backtracking Problems:

1. **Combinatorial Problems:** Permutations, combinations, subsets
2. **Constraint Satisfaction:** N-Queens, Sudoku
3. **Path Finding:** Maze solving, word search
4. **Decision Problems:** Satisfiability, coloring

When to Use Backtracking Use backtracking when: - You need to find all (or some) solutions to a problem - The problem involves making choices from a set of options - There are constraints that eliminate invalid solutions - You need to explore a search space systematically

Identification Criteria: - The problem asks for “all possible ways” or “all valid arrangements” - The problem involves arranging or selecting elements with constraints - You need to find combinations or permutations with specific properties - There’s a need to explore different paths in a search space - The problem can be solved by making a sequence of choices

Time and Space Complexity: - Time Complexity: Usually exponential, $O(k^n)$ where k is the branching factor - Space Complexity: Usually $O(n)$ for the recursion stack, plus storage for solutions

Example Problems

Easy: Letter Combinations of a Phone Number **Problem:** Given a string containing digits from 2-9, return all possible letter combinations that the number could represent (like on a telephone keypad).

Approach: Use backtracking to generate all combinations by making choices for each digit.

```
def letter_combinations(digits: str) -> list[str]:
    if not digits:
        return []

    # Mapping of digits to letters
    phone_map = {
        '2': 'abc', '3': 'def', '4': 'ghi', '5': 'jkl',
        '6': 'mno', '7': 'pqrs', '8': 'tuv', '9': 'wxyz'
    }

    result = []

    def backtrack(index: int, current: str) -> None:
        # If we've processed all digits, add the combination
        if index == len(digits):
            result.append(current)
            return

        # Get the letters for the current digit
```

```

letters = phone_map[digits[index]]

    # Try each letter
    for letter in letters:
        # Add the letter and recurse
        backtrack(index + 1, current + letter)

backtrack(0, "")
return result

```

Time Complexity: $O(4^n)$ where n is the number of digits (since a digit can map to at most 4 letters) Space Complexity: $O(n)$ for the recursion stack

Medium: Permutations Problem: Given an array `nums` of distinct integers, return all possible permutations.

Approach: Use backtracking to generate all permutations by choosing each element as the next in the sequence.

```

def permute(nums: list[int]) -> list[list[int]]:
    result = []
    n = len(nums)

    def backtrack(current: list[int], remaining: set[int]) -> None:
        # If permutation is complete
        if len(current) == n:
            result.append(current.copy())
            return

        # Try each remaining number
        for num in list(remaining):
            # Add the number to current permutation
            current.append(num)
            remaining.remove(num)

            # Recurse
            backtrack(current, remaining)

            # Backtrack (undo the choice)
            current.pop()
            remaining.add(num)

    backtrack([], set(nums))
    return result

```

Time Complexity: $O(n!)$ - We generate all permutations Space Complexity: $O(n)$ - For the recursion stack and to store each permutation

Hard: N-Queens Problem: The n-queens puzzle is the problem of placing n queens on an $n \times n$ chessboard such that no two queens attack each other. Given an integer n, return all distinct solutions to the n-queens puzzle.

Approach: Use backtracking to place queens one row at a time, checking that each new queen doesn't conflict with existing queens.

```
def solve_n_queens(n: int) -> list[list[str]]:
    result = []

    # Initialize board with empty cells
    board = [['.' ] * n for _ in range(n)]

    # Sets to track occupied columns and diagonals
    col_used = set()
    pos_diag_used = set() # r + c is constant
    neg_diag_used = set() # r - c is constant

    def backtrack(row: int) -> None:
        if row == n:
            # Found a valid solution, add to result
            solution = [''.join(row) for row in board]
            result.append(solution)
            return

        for col in range(n):
            # Check if position is valid (not under attack)
            if (col in col_used or
                row + col in pos_diag_used or
                row - col in neg_diag_used):
                continue

            # Place queen
            board[row][col] = 'Q'
            col_used.add(col)
            pos_diag_used.add(row + col)
            neg_diag_used.add(row - col)

            # Try next row
            backtrack(row + 1)

            # Backtrack (remove queen)
            board[row][col] = '.'
            col_used.remove(col)
            pos_diag_used.remove(row + col)
            neg_diag_used.remove(row - col)
```

```
backtrack(0)
return result
```

Time Complexity: $O(n!)$ - Though in practice it's better due to pruning
Space Complexity: $O(n^2)$ - For the board and sets to track attacks

Greedy Algorithms

Theory

Greedy algorithms make locally optimal choices at each step with the hope of finding a global optimum. Unlike dynamic programming, greedy algorithms do not reconsider earlier choices.

Key Principles:

1. **Greedy Choice Property:** A globally optimal solution can be reached by making locally optimal choices
2. **Optimal Substructure:** The optimal solution to the problem contains optimal solutions to subproblems

When Greedy Works: Greedy algorithms work when: - Making the locally optimal choice leads to a globally optimal solution - The problem has greedy choice property and optimal substructure - We can prove that the greedy approach works (usually by exchange argument)

Common Greedy Problems:

1. **Interval Scheduling:** Activity selection, meeting rooms
2. **Huffman Coding:** Data compression
3. **Minimum Spanning Tree:** Kruskal's and Prim's algorithms
4. **Shortest Path:** Dijkstra's algorithm (for non-negative weights)
5. **Coin Change:** With specific coin denominations (e.g., US coins)

When to Use Greedy Algorithms Use greedy when: - The problem asks for optimization (min/max) - You can make a locally optimal choice without reconsidering - You can prove that greedy leads to the global optimum

Identification Criteria: - The problem asks for minimum or maximum of something - Making a local optimal choice seems to lead to a global optimum - You can sort the input and process it in that order - The problem involves selecting a subset of items - The problem involves assigning resources optimally

Time and Space Complexity: - Time Complexity: Often $O(n \log n)$ due to sorting - Space Complexity: Usually $O(n)$ or $O(1)$ depending on implementation

Example Problems

Easy: Maximum Subarray Problem: Given an integer array `nums`, find the contiguous subarray which has the largest sum and return its sum.

Approach: Use Kadane's algorithm, a greedy approach that maintains the maximum sum ending at the current position.

```
def max_subarray(nums: list[int]) -> int:
    # Initialize current sum and max sum
    current_sum = 0
    max_sum = float('-inf')

    for num in nums:
        # Either start a new subarray or extend the existing one
        current_sum = max(num, current_sum + num)

        # Update the maximum sum seen so far
        max_sum = max(max_sum, current_sum)

    return max_sum
```

Time Complexity: $O(n)$ - We traverse the array once Space Complexity: $O(1)$ - We only use a constant amount of extra space

Medium: Jump Game Problem: Given an array of non-negative integers `nums`, you are initially positioned at the first index. Each element in the array represents your maximum jump length at that position. Determine if you are able to reach the last index.

Approach: Use a greedy approach by tracking the farthest position we can reach so far.

```
def can_jump(nums: list[int]) -> bool:
    # Maximum position we can reach so far
    max_reach = 0

    for i, jump_length in enumerate(nums):
        # If we can't reach the current position, return False
        if i > max_reach:
            return False

        # Update the maximum position we can reach
        max_reach = max(max_reach, i + jump_length)

    # If we can reach the last index, return True
    if max_reach >= len(nums) - 1:
        return True
```



```
return True # Will only reach here if nums has just one element
```

Time Complexity: $O(n)$ - We traverse the array once Space Complexity: $O(1)$ - We only use a constant amount of extra space

Hard: Minimum Number of Refueling Stops **Problem:** A car travels from a starting position to a destination which is `target` miles east of the starting position. There are gas stations along the way that can be used for refueling. The stations are represented as an array `stations` where `stations[i] = [position, fuel]` indicates that the `i`th gas station is `position` miles east of the starting position and has `fuel` liters of gas. The car starts with an initial tank of `startFuel` liters and can travel at most `startFuel` miles before refueling. Return the minimum number of refueling stops needed to reach the destination. Return -1 if it's not possible to reach the destination.

Approach: Use a greedy approach with a max-heap to keep track of the fuel at stations we've passed.

```
import heapq

def min_refuel_stops(target: int, start_fuel: int, stations: list[list[int]]) -> int:
    # Max heap to store fuel amounts (negated for max-heap in Python)
    fuel_heap = []

    # Initialize variables
    current_fuel = start_fuel
    current_position = 0
    num_stops = 0
    station_idx = 0

    # Continue until we reach the target or run out of fuel
    while current_position < target:
        # Calculate how far we can go with current fuel
        max_distance = current_position + current_fuel

        # If we can reach the target, return the number of stops
        if max_distance >= target:
            return num_stops

        # Add all stations we can reach to the heap
        while station_idx < len(stations) and stations[station_idx][0] <= max_distance:
            # Push negative fuel (for max-heap) to the heap
            heapq.heappush(fuel_heap, -stations[station_idx][1])
            station_idx += 1

    # If no stations available, we can't reach target
```

```

if not fuel_heap:
    return -1

# Use the station with the most fuel
most_fuel = -heapq.heappop(fuel_heap)

# Update position and fuel
current_position = max_distance
current_fuel = most_fuel
num_stops += 1

return num_stops

```

Time Complexity: $O(n \log n)$ - Where n is the number of stations (due to heap operations) Space Complexity: $O(n)$ - For the heap

Graphs

Theory

Graphs are collections of nodes (vertices) connected by edges. They are used to represent networks, relationships, paths, and many other structures.

Graph Types and Comparison:

Graph Type	Description	Key Properties	Common Applications
Undirected Graph	Edges have no direction	Symmetric relationships	Social networks, road maps
Directed Graph (Digraph)	Edges have direction	Asymmetric relationships	Web pages, dependencies
Weighted Graph	Edges have weights/costs	Quantified relationships	Road networks, flow networks
Unweighted Graph	Edges have no weights	Binary relationships	Simple connections
Cyclic Graph	Contains at least one cycle	Can have loops	Network topologies
Acyclic Graph	Contains no cycles	No paths return to same vertex	Dependency hierarchies
Connected Graph	All vertices are reachable	No isolated components	Robust networks
Disconnected Graph	Has isolated components	Multiple connected components	Fragmented systems

Graph Type	Description	Key Properties	Common Applications
Complete Graph	Every vertex connected to all others	Maximum number of edges	Fully meshed networks
Bipartite Graph	Vertices in two groups, edges only between groups	Two-colorable	Matching problems

Graph Representation:

1. Adjacency Matrix:

```
# n vertices
graph = [[0] * n for _ in range(n)]
# Edge from i to j with weight w
graph[i][j] = w
```

2. Adjacency List:

```
# n vertices
graph = [[] for _ in range(n)]
# Edge from i to j with weight w
graph[i].append((j, w))
```

Graph Algorithms Comparison:

Algorithm	Purpose	Time Complexity	Space Complexity	Key Features
BFS	Shortest path in un-weighted graph	$O(V+E)$	$O(V)$	Level-order traversal, queue-based
DFS	Explore all paths, detect cycles	$O(V+E)$	$O(V)$	Deep exploration, stack-based
Dijkstra's	Shortest path with positive weights	$O(E \log V)$	$O(V)$	Greedy, uses priority queue

Algorithm	Purpose	Time Complexity	Space Complexity	Key Features
Bellman-Ford	Shortest path with negative weights	$O(VE)$	$O(V)$	Can detect negative cycles
Floyd-Warshall	All-pairs shortest paths	$O(V^3)$	$O(V^2)$	Works with negative weights
Kruskal's	Minimum spanning tree	$O(E \log E)$	$O(V)$	Greedy, uses union-find
Prim's	Minimum spanning tree	$O(E \log V)$	$O(V)$	Greedy, similar to Dijkstra's
Topological Sort	Order vertices in DAG	$O(V+E)$	$O(V)$	Works only on DAGs

Graph Traversals:

1. Breadth-First Search (BFS):

```

from collections import deque

def bfs(graph, start):
    visited = set([start])
    queue = deque([start])
    result = []

    while queue:
        vertex = queue.popleft()
        result.append(vertex)

        for neighbor in graph[vertex]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)

    return result

```

2. Depth-First Search (DFS):

```
def dfs(graph, start):
    visited = set()
    result = []

    def dfs_recursive(vertex):
        visited.add(vertex)
        result.append(vertex)

        for neighbor in graph[vertex]:
            if neighbor not in visited:
                dfs_recursive(neighbor)

    dfs_recursive(start)
    return result
```

When to Use Graphs Use graphs when: - The problem involves relationships between entities - You need to find paths, connections, or traversals - The problem can be modeled as nodes and edges - You need to analyze network properties or connectivity

Identification Criteria: - The problem mentions networks, connections, or relationships - You need to find paths, cycles, or connected components - The problem involves traversal or search in a network - The problem can be visualized as points connected by lines - You need to find the shortest path, minimum spanning tree, etc.

Time and Space Complexity: - Space Complexity: $O(V+E)$ to represent the graph - Time Complexity: Varies by algorithm (see comparison table)

Example Problems

Easy: Number of Islands Problem: Given an $m \times n$ 2D binary grid grid which represents a map of '1's (land) and '0's (water), return the number of islands. An island is surrounded by water and is formed by connecting adjacent lands horizontally or vertically.

Approach: Use DFS to explore each island. When we find a land cell, explore all connected land cells and mark them as visited.

```
def num_islands(grid: list[list[str]]) -> int:
    if not grid or not grid[0]:
        return 0

    m, n = len(grid), len(grid[0])
    count = 0

    def dfs(row: int, col: int) -> None:
        # Check if out of bounds or not land
```

```

    if (row < 0 or row >= m or col < 0 or col >= n or
        grid[row][col] != '1'):
        return

    # Mark as visited
    grid[row][col] = '0'

    # Explore neighbors
    dfs(row + 1, col)
    dfs(row - 1, col)
    dfs(row, col + 1)
    dfs(row, col - 1)

    # Iterate through each cell in the grid
    for i in range(m):
        for j in range(n):
            if grid[i][j] == '1':
                count += 1 # Found an island
                dfs(i, j) # Explore the island

    return count

```

Time Complexity: $O(mn)$ - We visit each cell at most once Space Complexity: $O(mn)$ - In the worst case, the entire grid is an island (for the recursion stack)

Medium: Course Schedule Problem: There are a total of numCourses courses you have to take, labeled from 0 to numCourses-1. Some courses may have prerequisites. Given the total number of courses and a list of prerequisite pairs, is it possible for you to finish all courses?

Approach: Model the problem as a directed graph and check for cycles using DFS. If there's a cycle, it's impossible to finish all courses.

```

def can_finish(num_courses: int, prerequisites: list[list[int]]) -> bool:
    # Build adjacency list
    graph = [[] for _ in range(num_courses)]
    for course, prereq in prerequisites:
        graph[prereq].append(course)

    # Keep track of visited nodes
    # 0 = unvisited, 1 = visiting (in current path), 2 = visited (completed)
    visited = [0] * num_courses

    def has_cycle(node: int) -> bool:
        # Already fully visited this node
        if visited[node] == 2:
            return False

```

```

    # Already visiting this node - cycle detected
    if visited[node] == 1:
        return True

    # Mark as visiting
    visited[node] = 1

    # Check neighbors
    for neighbor in graph[node]:
        if has_cycle(neighbor):
            return True

    # Mark as visited
    visited[node] = 2
    return False

# Check each course
for course in range(num_courses):
    if has_cycle(course):
        return False

return True

```

Time Complexity: $O(V+E)$ - Where V is the number of courses and E is the number of prerequisites Space Complexity: $O(V+E)$ - For the graph and visited array

Hard: Network Delay Time Problem: You are given a network of n nodes, labeled from 1 to n. You are also given times, a list of travel times as directed edges $times[i] = (u, v, w)$, where u is the source node, v is the target node, and w is the time it takes for a signal to travel from source to target. We will send a signal from a given node k. Return the time it takes for all the n nodes to receive the signal. If it is impossible for all the n nodes to receive the signal, return -1.

Approach: Use Dijkstra's algorithm to find the shortest path from the source to all nodes.

```

import heapq

def network_delay_time(times: list[list[int]], n: int, k: int) -> int:
    # Build adjacency list
    graph = [[] for _ in range(n + 1)]
    for u, v, w in times:
        graph[u].append((v, w))

    # Dijkstra's algorithm

```

```

distances = [float('inf')] * (n + 1)
distances[0] = 0 # Node 0 doesn't exist
distances[k] = 0 # Starting node

# Priority queue for Dijkstra's
pq = [(0, k)] # (distance, node)

while pq:
    dist, node = heapq.heappop(pq)

    # If we've found a longer path, skip
    if dist > distances[node]:
        continue

    # Check neighbors
    for neighbor, weight in graph[node]:
        if dist + weight < distances[neighbor]:
            distances[neighbor] = dist + weight
            heapq.heappush(pq, (distances[neighbor], neighbor))

# Find the maximum distance (time for signal to reach all nodes)
max_dist = max(distances[1:])

return max_dist if max_dist < float('inf') else -1

```

Time Complexity: $O(E \log V)$ - Where E is the number of edges and V is the number of nodes
 Space Complexity: $O(V+E)$ - For the graph, distances, and priority queue

Intervals

Theory

Intervals are pairs of values that represent a range. They are commonly used to model time periods, numerical ranges, and overlapping segments.

Key Concepts:

1. **Interval Representation:** Most commonly represented as $[start, end]$
2. **Interval Overlap:** Two intervals overlap if one's start is less than the other's end and vice versa
3. **Interval Merging:** Combining overlapping intervals
4. **Interval Intersection:** Finding the common range between intervals

Common Interval Operations:

1. **Sort Intervals:** Usually sorted by start time

2. **Check Overlap:** $\max(a[0], b[0]) \leq \min(a[1], b[1])$
3. **Merge Intervals:** Combine overlapping intervals into one
4. **Find Gaps:** Identify spaces between non-overlapping intervals

When to Use Interval Algorithms Use interval algorithms when: - The problem involves ranges, segments, or periods - You need to find overlaps, merges, or intersections - The problem deals with scheduling or resource allocation - You need to track coverage or gaps

Identification Criteria: - The problem mentions ranges, intervals, or segments - The problem involves overlapping regions - The problem deals with scheduling, booking, or reservations - The problem can be visualized as line segments on a number line - You need to find the maximum number of overlapping intervals

Time and Space Complexity: - Time Complexity: Usually $O(n \log n)$ due to sorting - Space Complexity: Usually $O(n)$ to store the intervals or result

Example Problems

Easy: Meeting Rooms **Problem:** Given an array of meeting time intervals where $\text{intervals}[i] = [\text{start}_i, \text{end}_i]$, determine if a person could attend all meetings.

Approach: Sort the intervals by start time and check if any meeting overlaps with the next one.

```
def can_attend_meetings(intervals: list[list[int]]) -> bool:
    # Sort intervals by start time
    intervals.sort(key=lambda x: x[0])

    # Check for overlaps
    for i in range(1, len(intervals)):
        if intervals[i][0] < intervals[i-1][1]:
            return False # Overlap found

    return True # No overlaps
```

Time Complexity: $O(n \log n)$ - Dominated by the sorting operation
Space Complexity: $O(1)$ - If we don't count the input, we use constant extra space

Medium: Merge Intervals **Problem:** Given an array of intervals where $\text{intervals}[i] = [\text{start}_i, \text{end}_i]$, merge all overlapping intervals and return an array of the non-overlapping intervals that cover all the intervals in the input.

Approach: Sort the intervals by start time, then iterate through them, merging overlapping intervals.

```
def merge(intervals: list[list[int]]) -> list[list[int]]:
    # Sort intervals by start time
```

```

intervals.sort(key=lambda x: x[0])

merged = []

for interval in intervals:
    # If merged is empty or no overlap with last merged interval
    if not merged or merged[-1][1] < interval[0]:
        merged.append(interval)
    else:
        # Overlapping intervals - merge them
        merged[-1][1] = max(merged[-1][1], interval[1])

return merged

```

Time Complexity: $O(n \log n)$ - Dominated by the sorting operation
Space Complexity: $O(n)$ - For the output array

Hard: Insert Interval Problem: You are given an array of non-overlapping intervals `intervals` where `intervals[i] = [start_i, end_i]` and intervals are sorted in ascending order by `start_i`. You are also given an interval `newInterval = [start, end]`. Insert `newInterval` into `intervals` such that `intervals` is still sorted in ascending order by `start_i` and `intervals` still does not have any overlapping intervals. Return `intervals` after the insertion.

Approach: Process intervals in three groups: those that come before the new interval, those that overlap with it, and those that come after it.

```

def insert(intervals: list[list[int]], new_interval: list[int]) -> list[list[int]]:
    result = []
    i = 0
    n = len(intervals)

    # Add intervals that come before new_interval
    while i < n and intervals[i][1] < new_interval[0]:
        result.append(intervals[i])
        i += 1

    # Merge overlapping intervals
    merged_interval = new_interval.copy()
    while i < n and intervals[i][0] <= new_interval[1]:
        merged_interval[0] = min(merged_interval[0], intervals[i][0])
        merged_interval[1] = max(merged_interval[1], intervals[i][1])
        i += 1

    result.append(merged_interval)

    # Add intervals that come after the merged interval

```

```

while i < n:
    result.append(intervals[i])
    i += 1

return result

```

Time Complexity: $O(n)$ - We process each interval once
 Space Complexity: $O(n)$
 - For the result array

Math and Geometry

Theory

Math and geometry problems require understanding mathematical concepts and spatial relationships to solve algorithmic challenges.

Key Mathematical Concepts:

1. **Number Theory:** Prime numbers, divisibility, GCD/LCM
2. **Combinatorics:** Permutations, combinations, counting
3. **Probability:** Expected values, conditional probability
4. **Algebra:** Equations, polynomials, series
5. **Geometry:** Points, lines, shapes, angles, distances

Common Math Functions:

```

# Greatest Common Divisor (GCD)
def gcd(a: int, b: int) -> int:
    while b:
        a, b = b, a % b
    return a

# Least Common Multiple (LCM)
def lcm(a: int, b: int) -> int:
    return a * b // gcd(a, b)

# Prime Check
def is_prime(n: int) -> bool:
    if n <= 1:
        return False
    if n <= 3:
        return True
    if n % 2 == 0 or n % 3 == 0:
        return False
    i = 5
    while i * i <= n:
        if n % i == 0 or n % (i + 2) == 0:

```

```

        return False
    i += 6
    return True

# Power with modulo
def power_mod(x: int, y: int, m: int) -> int:
    res = 1
    x = x % m
    while y > 0:
        if y & 1:
            res = (res * x) % m
        y = y >> 1
        x = (x * x) % m
    return res

```

Geometric Utilities:

```

# Euclidean distance between two points
def distance(p1: tuple[float, float], p2: tuple[float, float]) -> float:
    return ((p1[0] - p2[0])**2 + (p1[1] - p2[1])**2) ** 0.5

# Check if points are collinear
def collinear(p1, p2, p3) -> bool:
    return (p2[1] - p1[1]) * (p3[0] - p2[0]) == (p3[1] - p2[1]) * (p2[0] - p1[0])

# Area of a triangle
def triangle_area(p1, p2, p3) -> float:
    return 0.5 * abs(p1[0]*(p2[1]-p3[1]) + p2[0]*(p3[1]-p1[1]) + p3[0]*(p1[1]-p2[1]))

```

When to Use Math and Geometry Use mathematical approaches when: - The problem involves numbers, counting, or probability - The problem requires spatial reasoning or geometric calculations - You need to optimize based on mathematical properties - The problem involves divisibility, primes, or modular arithmetic

Identification Criteria: - The problem involves calculations, equations, or numerical patterns - The problem deals with points, lines, shapes, or spatial relationships - The problem requires counting arrangements or combinations - The problem involves probabilities or expected values - The problem can be solved with number theory concepts

Time and Space Complexity: - Varies widely depending on the specific problem and approach

Example Problems

Easy: Count Primes Problem: Count the number of prime numbers less than a non-negative number n .

Approach: Use the Sieve of Eratosthenes to efficiently count primes.

```
def count_primes(n: int) -> int:
    if n <= 2:
        return 0

    # Initialize a list to track prime numbers
    is_prime = [True] * n
    is_prime[0] = is_prime[1] = False

    # Apply Sieve of Eratosthenes
    for i in range(2, int(n**0.5) + 1):
        if is_prime[i]:
            # Mark all multiples of i as non-prime
            for j in range(i*i, n, i):
                is_prime[j] = False

    # Count primes
    return sum(is_prime)
```

Time Complexity: $O(n \log \log n)$ - From Sieve of Eratosthenes Space Complexity: $O(n)$ - For the `is_prime` array

Medium: Pow(x, n) Problem: Implement `pow(x, n)`, which calculates x raised to the power n .

Approach: Use fast power algorithm (binary exponentiation) to efficiently calculate the power.

```
def my_pow(x: float, n: int) -> float:
    # Handle negative exponent
    if n < 0:
        x = 1 / x
        n = -n

    # Binary exponentiation
    result = 1
    current_product = x

    while n > 0:
        # If n is odd, multiply result by current_product
        if n % 2 == 1:
            result *= current_product
```

```

        # Square the current_product
        current_product *= current_product

        # Integer division by 2
        n //= 2

    return result

```

Time Complexity: $O(\log n)$ - We halve n in each iteration Space Complexity: $O(1)$ - We use a constant amount of extra space

Hard: Max Points on a Line Problem: Given an array of points where $\text{points}[i] = [x_i, y_i]$ represents a point on the X-Y plane, return the maximum number of points that lie on the same straight line.

Approach: For each point, calculate the slope with every other point and count points with the same slope.

```

from collections import defaultdict
import math

def max_points(points: list[list[int]]) -> int:
    n = len(points)
    if n <= 2:
        return n

    max_count = 0

    for i in range(n):
        # Count points with the same slope through points[i]
        slope_count = defaultdict(int)
        duplicate = 0 # Count points at the same position as points[i]

        for j in range(i + 1, n):
            x1, y1 = points[i]
            x2, y2 = points[j]

            # Check for duplicate points
            if x1 == x2 and y1 == y2:
                duplicate += 1
                continue

            # Calculate the slope
            if x1 == x2: # Vertical line
                slope = float('inf')
            else:

```

```

        # Use GCD to get the reduced form of the slope
        dx = x2 - x1
        dy = y2 - y1
        g = math.gcd(abs(dx), abs(dy))

        # Handle signs properly
        if dx < 0:
            dx, dy = -dx, -dy

        slope = (dy // g, dx // g)

        # Count points with this slope
        slope_count[slope] += 1

        # Find maximum number of points on the same line through points[i]
        current_max = 0
        for count in slope_count.values():
            current_max = max(current_max, count)

        # Add 1 for points[i] itself and any duplicates
        max_count = max(max_count, current_max + duplicate + 1)

    return max_count

```

Time Complexity: $O(n^2)$ - We check each pair of points
 Space Complexity: $O(n)$
 - For storing the slope counts

Trie Data Structure

Theory

A Trie (pronounced “try”) is a tree-like data structure used to store a collection of strings. It’s particularly useful for efficient string operations like prefix lookups.

Key Properties:

1. **Structure:** Each node represents a character or a complete word
2. **Path:** Following a path from the root forms a string
3. **Shared Prefixes:** Strings with common prefixes share the same initial nodes
4. **End Markers:** Special markers indicate complete words

Basic Trie Implementation:

```

class TrieNode:
    def __init__(self):
        self.children = {} # Map from character to TrieNode

```

```

        self.is_end_of_word = False

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word: str) -> None:
        node = self.root
        for char in word:
            if char not in node.children:
                node.children[char] = TrieNode()
            node = node.children[char]
        node.is_end_of_word = True

    def search(self, word: str) -> bool:
        node = self.root
        for char in word:
            if char not in node.children:
                return False
            node = node.children[char]
        return node.is_end_of_word

    def starts_with(self, prefix: str) -> bool:
        node = self.root
        for char in prefix:
            if char not in node.children:
                return False
            node = node.children[char]
        return True

```

Key Operations and Complexity:

- Insert: $O(m)$ where m is the length of the word
- Search: $O(m)$ where m is the length of the word
- Prefix Search: $O(m)$ where m is the length of the prefix
- Space Complexity: $O(n*m)$ where n is the number of words and m is the average length

When to Use Tries Use Tries when: - You need efficient prefix searching
 - You're working with a dictionary or word list - You need to validate strings against a vocabulary - You're implementing autocomplete or spell checking - You need to find words with common prefixes

Identification Criteria: - The problem involves searching for words or prefixes in a collection - You need to efficiently check if a string belongs to a set - The problem mentions autocomplete, word validation, or dictionary lookup - You need

to find all words with a specific prefix - The problem involves string matching or pattern searching

Time and Space Complexity: - Time Complexity: Most operations are $O(m)$ where m is the length of the string - Space Complexity: $O(n*m)$ where n is the number of strings and m is the average length

Example Problems

Easy: Implement Trie (Prefix Tree) Problem: Implement a trie with insert, search, and startsWith methods.

Approach: Use the basic trie implementation shown above.

```
class TrieNode:
    def __init__(self):
        self.children = {}
        self.is_end_of_word = False

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word: str) -> None:
        node = self.root
        for char in word:
            if char not in node.children:
                node.children[char] = TrieNode()
            node = node.children[char]
        node.is_end_of_word = True

    def search(self, word: str) -> bool:
        node = self.root
        for char in word:
            if char not in node.children:
                return False
            node = node.children[char]
        return node.is_end_of_word

    def starts_with(self, prefix: str) -> bool:
        node = self.root
        for char in prefix:
            if char not in node.children:
                return False
            node = node.children[char]
        return True
```

Time Complexity: - insert: $O(m)$ where m is the length of the word - search:

$O(m)$ where m is the length of the word - starts_with: $O(m)$ where m is the length of the prefix Space Complexity: $O(n*m)$ where n is the number of words and m is the average length

Medium: Design Add and Search Words Data Structure Problem:

Design a data structure that supports adding new words and finding if a string matches any previously added string. The search function can include dots '.' which can match any letter.

Approach: Extend the basic trie to handle the wildcard character during search.

```
class TrieNode:
    def __init__(self):
        self.children = {}
        self.is_end_of_word = False

class WordDictionary:
    def __init__(self):
        self.root = TrieNode()

    def add_word(self, word: str) -> None:
        node = self.root
        for char in word:
            if char not in node.children:
                node.children[char] = TrieNode()
            node = node.children[char]
        node.is_end_of_word = True

    def search(self, word: str) -> bool:
        def dfs(node: TrieNode, index: int) -> bool:
            # Base case: reach the end of the word
            if index == len(word):
                return node.is_end_of_word

            char = word[index]

            # Handle wildcard
            if char == '.':
                # Try all possible characters
                for child in node.children.values():
                    if dfs(child, index + 1):
                        return True
                return False

            # Normal character
            if char not in node.children:
```

```

        return False

    return dfs(node.children[char], index + 1)

return dfs(self.root, 0)

```

Time Complexity: - add_word: $O(m)$ where m is the length of the word -
 search: $O(n26^d)$ where n is the number of words, d is the number of dots Space
 Complexity: $O(nm)$ where n is the number of words and m is the average length

Hard: Word Search II Problem: Given an $m \times n$ board of characters and a list of strings words, return all words on the board. Each word must be constructed from letters of sequentially adjacent cells, where adjacent cells are horizontally or vertically neighboring. The same letter cell may not be used more than once in a word.

Approach: Build a trie from the word list, then use backtracking to explore the board and find all words.

```

class TrieNode:
    def __init__(self):
        self.children = {}
        self.word = None # Store the complete word at leaf nodes

def find_words(board: list[list[str]], words: list[str]) -> list[str]:
    # Build Trie
    root = TrieNode()
    for word in words:
        node = root
        for char in word:
            if char not in node.children:
                node.children[char] = TrieNode()
            node = node.children[char]
        node.word = word

    # Define search function
    def dfs(row: int, col: int, node: TrieNode, result: list[str]) -> None:
        # Check bounds and if character matches any child
        if (row < 0 or row >= len(board) or col < 0 or col >= len(board[0]) or
            board[row][col] not in node.children):
            return

        char = board[row][col]
        node = node.children[char]

        # If we reached a word, add it to result
        if node.word:

```

```

        result.append(node.word)
        node.word = None # Avoid duplicates

    # Mark cell as visited
    board[row][col] = '#'

    # Explore neighbors
    for dr, dc in [(0, 1), (1, 0), (0, -1), (-1, 0)]:
        dfs(row + dr, col + dc, node, result)

    # Restore cell
    board[row][col] = char

# Search the board
result = []
for row in range(len(board)):
    for col in range(len(board[0])):
        dfs(row, col, root, result)

return result

```

Time Complexity: $O(mn4^L)$ where $m*n$ is the board size and L is the maximum word length
 Space Complexity: $O(\sum(L_i))$ where L_i is the length of each word in the input

SQL for Analytics

Example Problems

Medium: Department Top Three Salaries **Problem:** Write a SQL query to find employees who earn the top three salaries in each department.

```

SELECT d.Name AS Department, e.Name AS Employee, e.Salary
FROM Employee e
JOIN Department d ON e.DepartmentId = d.Id
WHERE (
    SELECT COUNT(DISTINCT e2.Salary)
    FROM Employee e2
    WHERE e2.Salary > e.Salary AND e2.DepartmentId = e.DepartmentId
) < 3
ORDER BY d.Name, e.Salary DESC;

```

Medium: Consecutive Numbers **Problem:** Write a SQL query to find all numbers that appear at least three times consecutively.

```

SELECT DISTINCT l1.Num AS ConsecutiveNums
FROM Logs l1

```

```
JOIN Logs 12 ON 11.Id = 12.Id - 1 AND 11.Num = 12.Num
JOIN Logs 13 ON 12.Id = 13.Id - 1 AND 12.Num = 13.Num;
```

Medium: Nth Highest Salary Problem: Write a SQL function to get the nth highest salary from the Employee table.

```
CREATE FUNCTION getNthHighestSalary(N INT) RETURNS INT
BEGIN
  SET N = N - 1;
  RETURN (
    SELECT DISTINCT Salary
    FROM Employee
    ORDER BY Salary DESC
    LIMIT 1 OFFSET N
  );
END
```

Advanced: Human Traffic of Stadium Problem: Write a SQL query to display the records with three or more consecutive rows where the amount of people is greater than or equal to 100.

```
SELECT DISTINCT s1.*
FROM Stadium s1
JOIN Stadium s2 ON ABS(s1.id - s2.id) <= 2
JOIN Stadium s3 ON ABS(s1.id - s3.id) <= 2 AND ABS(s2.id - s3.id) <= 2
WHERE s1.people >= 100 AND s2.people >= 100 AND s3.people >= 100
      AND s1.id != s2.id AND s1.id != s3.id AND s2.id != s3.id
ORDER BY s1.id;
```

Advanced: Market Analysis Problem: Write a SQL query to find for each user, the join date and the number of orders they made as a buyer in 2019.

```
SELECT u.user_id AS buyer_id, u.join_date,
       COUNT(CASE WHEN YEAR(o.order_date) = 2019 THEN 1 ELSE NULL END) AS orders_in_2019
FROM Users u
LEFT JOIN Orders o ON u.user_id = o.buyer_id
GROUP BY u.user_id, u.join_date
ORDER BY u.user_id;
```

Advanced: Tournament Winners Problem: Write a SQL query to find the winner in each group.

```
WITH PlayerScores AS (
  SELECT player_id, group_id, SUM(score) AS total_score
  FROM (
    SELECT first_player AS player_id, first_score AS score
    FROM Matches m
```

```

        JOIN Players p ON m.first_player = p.player_idgg

    UNION ALL

    SELECT second_player, second_score
    FROM Matches m
    JOIN Players p ON m.second_player = p.player_id
) AS scores
JOIN Players p ON scores.player_id = p.player_id
GROUP BY player_id, group_id
),
RankedScores AS (
    SELECT player_id, group_id, total_score,
           RANK() OVER (PARTITION BY group_id ORDER BY total_score DESC, player_id) AS rnk
    FROM PlayerScores
)
SELECT group_id, player_id
FROM RankedScores
WHERE rnk = 1
ORDER BY group_id;

```